Fallstudie

Faktura — vom Hochschulprojekt zur auslieferbaren Fakturierungsanwendung

Lucas Strubel

Version 1.2 · 10.10.2026

Inhaltsverzeichnis

1 Dokumentenhistorie **2**

2 Überblick **2**

3 Das Problem **2**
3.1 Fakturierung ist ein Compliance-Problem im CRUD-Gewand 2
3.2 Ein abgenommenes Projekt ist noch kein Produkt 3

4 Die Lösung **3**
4.1 Lückenlosigkeit ist eine Eigenschaft der *Vergabe*, nicht der sichtbaren Nummern 4
4.2 Versendete Belege sind unveränderlich, korrigiert wird durch neue Belege 4
4.3 Belege speichern Kunde, Aussteller und Preise als Snapshot 5
4.4 PDF und E-Rechnung müssen auf den Cent übereinstimmen 5
4.5 Die Oberfläche erfährt nur von Daten, die den Commit überlebt haben 5

5 Vorgehen **6**

6 Technologiewahl und Begründung **6**

7 Ergebnisse **7**

8 Was ich gelernt habe **8**
8.1 Arbeiten mit Claude Code 9

9 Ausblick **9**

10 Ergänzung 3.1.0-rc.1 (10.10.2026) **9**
10.1 Ergänzende Review- und Portfolio-Arbeit mit Codex 10

1 Dokumentenhistorie

Version	Datum	Grund der Änderung
1.0	29.07.2026	Erstfassung: Fallstudie zur Weiterentwicklung vom Hochschulprojekt zum Produkt
1.1	24.09.2026	Abschluss der Version 3.0: Steuer je Steuersatz, Stornorechnung, Zahlungseingang, Aussteller-Snapshot; deutsche Fassung ist die einzige Fassung
1.2	10.10.2026	Review-Remediation 3.1.0-rc.1; historische Nachweise und offene Freigabeschranken getrennt

2 Überblick

Gegenstand	Faktura — lokal betriebene Fakturierungsanwendung für Freiberufler und Kleinstunternehmen
Ursprung	Gestartet im Rahmen des Moduls *Software Engineering 1* an der TH Mannheim (Sommersemester 2026): 12 Studierende in vier Gruppen à drei, je Gruppe eine Komponente (A–D)
Rolle	Version 1.0: Verwaltung des Repositorys und Verfassen sämtlicher Projektdokumente (Lastenheft, Pflichtenheft, Modultestplan und -bericht). Version 2.0 und 3.0: Neubau und Weiterentwicklung als Einzelentwickler
Umfang	Java-Domänenmodell und Services, JavaFX/FXML-Oberfläche, sechs Flyway-Migrationen, automatisierte Regressionen und deutschsprachige Spezifikation
Technik	Java 21 · Spring Boot · JavaFX · SQLite + Flyway · PDFBox · Mustang (EN 16931) · Maven · JUnit 5
Stand	RC 3.1.0-rc.1 unveröffentlicht; aktuelle Nachweise und ausstehende Freigaben im Modultestbericht

3 Das Problem

Das Projekt enthält zwei Probleme, die aufeinander liegen. Das zweite ist das interessantere.

3.1 Fakturierung ist ein Compliance-Problem im CRUD-Gewand

Rechnungssoftware muss Beträge, Nummern und historische Angaben nachvollziehbar halten. Faktura setzt dafür transaktionale Nummernvergabe, Änderungssperren und gespeicherte Kunden-, Aussteller-, Einheiten- und Betragssnapshots ein. Die lückenlose Nummernfolge ist eine bewusst streng gewählte Projektinvariante. Diese Kontrollen ersetzen weder eine rechtliche Prüfung noch organisatorische Aufbewahrungs- und Verfahrensregeln. Aus dem Speichermedium oder Betriebsmodell allein folgt keine Rechtssicherheit.

Faktura ergänzt die PDF-Ausgabe um strukturiertes EN-16931-CII-XML. Der Export muss dieselben Beträge und Einheiten wie der gespeicherte Beleg wiedergeben. Die Prüfung ist eine technische Validierung dieses Formats und keine rechtliche Gesamtbewertung.

Das macht die Fachlichkeit trügerisch schwer. Die Masken sehen aus wie gewöhnliches CRUD, darunter liegen jedoch Regeln, die **immer** gelten müssen, nicht meistens:

Regel	Was sie tatsächlich verlangt
Projektinvariante GR-01	Eine fehlgeschlagene Speicherung verbraucht keine Nummer
Änderungssperre GR-02	Eine versendete Rechnung wird durch eine Stornorechnung ausgeglichen
§ 14 UStG	Jede Rechnung trägt einen festen Satz Pflichtangaben — darunter Steuernummer oder USt-IdNr. und die Steuer je Steuersatz
EN 16931	Strukturiertes XML zusätzlich zur menschenlesbaren PDF-Fassung — mit denselben Beträgen
Lokale Datenhaltung	Kundendaten bleiben auf dem Rechner; dies allein belegt keine vollständige DSGVO-Konformität

3.2 Ein abgenommenes Projekt ist noch kein Produkt

Version 1.0 leistete, was ein Lehrprojekt leisten soll. Sie durchlief den vollständigen V-Modell-Prozess — Lastenheft, Pflichtenheft, Entwurf, Modultestplan, Abnahme — und bestand ihn: alle Muss-Anforderungen umgesetzt, 71 von 71 Modultestfällen grün.

Auslieferbar war sie damit nicht:

- Die Daten lagen in **JSON-Dateien**, die bei jeder Änderung vollständig neu geschrieben wurden.
- Der Rechnungszähler war eine **Zahl im Speicher** und wurde *vor* dem Speichern erhöht. Schlug das Speichern fehl, war die Nummer verbraucht — ein stiller Verstoß gegen genau die Regel, auf die es fachlich am meisten ankommt.
- Die Verdrahtung war **manuelle Dependency Injection** in `Main.java`.
- Die Oberfläche war **Swing**, und jede länger laufende Aktion — PDF-Export, CSV, Druck — lief im Ereignis-Thread; das Fenster fror ein.
- „Installation“ hieß: JDK vorhanden, JAR starten.
- Die E-Rechnung, also das gesetzlich Geforderte, war ausdrückliches Nichtziel.

Das eigentliche Projekt war das zweite: eine Codebasis, die *funktioniert*, so umzubauen, dass sie *trägt* — ohne die Teile wegzuwerfen, die ihren Platz verdient haben.

4 Die Lösung

Faktura bildet den vollständigen kaufmännischen Dokumentenzyklus ab — Angebot → Auftragsbestätigung → Lieferschein → Rechnung —, jeder Beleg aus seinem Vorgänger ableitbar, mit PDF- und EN-16931-Ausgabe, Zahlungseingang, Stornorechnung, Kunden- und Produktverwaltung sowie einem Firmenprofil, das den Briefkopf speist.

Die Architektur behält die Vier-Komponenten-Teilung der ursprünglichen Spezifikation bei — sie entspricht den vier Gruppen des Studierendenteams, und die Anforderungs-IDs der Spezifikationen sind darauf abgebildet. Diese Nachvollziehbarkeit ist erhaltenswert:

Paket	Komponente	Verantwortung
`dokumente`	A	Dokumentenzyklus, Belegnummern, Zahlungseingang, Stornorechnung, PDF-Export, E-Rechnung EN 16931
`produkte`	B	Produktverwaltung (CRUD, Nummernvergabe, Löschsperre)
`kunden`	C	Kundenverwaltung (CRUD, Nummernvergabe, Löschsperre)
`gui`	D	JavaFX-Oberfläche (FXML-Ansichten, Spring-injizierte Controller, Rechnungswizard)
`gemeinsam`	—	Ereignisbus, Validierung, CSV, Datensicherung, Nummernkreise, Ausnahmen
`firma`	—	Firmenprofil (Briefkopf, Steuerkennung, Bankverbindung)

Fünf Entscheidungen tragen die regulatorischen Regeln. Bei jeder ist die naheliegende Umsetzung die falsche.

4.1 Lückenlosigkeit ist eine Eigenschaft der *Vergabe*, nicht der sichtbaren Nummern

Der Zähler aus Version 1.0 lieferte lückenlose Nummern — bis etwas fehlschlug. Die Lösung bestand darin, die Nummernvergabe nicht länger als Buchführung der Anwendung zu behandeln, sondern als Datenbank-Invariante: eine Tabelle `nummernkreis` (Flyway-Migration V3), gelesen und erhöht über `JdbcNummernkreis`, ausgeführt **innerhalb derselben Transaktion** wie das Einfügen des Belegs. Ein fehlgeschlagenes Speichern rollt die Vergabe mit zurück.

Den Nachweis führt ein Test, der den Fehler erzwingt: `NummernkreisIntegritaetTest` schiebt ein `@Primary`-Repository davor, das auf Kommando eine Ausnahme wirft, und prüft anschließend, dass die nächste erfolgreiche Rechnung die laufende Nummer 2 trägt und nicht 3 — und dass der gescheiterte Beleg nirgends persistiert wurde. Ein weiterer Testfall prüft, dass der Zähler einen Neustart übersteht; „lückenlos“ überdauert den Prozess.

Zusätzlich lehnt die Fachlogik ab, eine bereits vergebene Nummer ein zweites Mal zu verwenden: Liefert der Nummernkreis — etwa nach einer beschädigten Übernahme — eine Nummer, die schon existiert, wird abgebrochen, statt eine versendete Rechnung stillschweigend zu überschreiben.

4.2 Versendete Belege sind unveränderlich, korrigiert wird durch neue Belege

`Dokument.pruefeAenderbar()` weist jede inhaltliche Änderung ab, sobald ein Beleg den Status `VERSENDET` oder `STORNIERT` erreicht hat. An die Stelle des Löschens tritt die Stornierung. Eine noch nicht versendete Rechnung wird nur gekennzeichnet; eine versendete liegt beim Kunden und wird durch eine **Stornorechnung** ausgeglichen — eine neue Rechnung mit negativen Mengen, eigener Nummer aus dem lückenlosen Rechnungskreis und Verweis auf das Original, als E-Rechnung ausgegeben mit der Dokumentart 384 (korrigierte Rechnung).

Der Zahlungseingang ist dagegen *keine* inhaltliche Änderung: Er wird als eigenes Datum erfasst und ist auch nach dem Versand zulässig. Ihn als fünften Status zu modellieren, hätte die Statusfolge und die Datenbankprüfung aufgebrochen, ohne fachlich etwas zu gewinnen.

Eine Feinheit zeigt sich erst in der Praxis: Beim Laden eines versendeten Belegs aus der Datenbank muss der Status **zuletzt** gesetzt werden, sonst weist die Prüfung genau die Daten ab, die sie schützen soll. Und seine Summen werden *gelesen*, nicht neu berechnet — sonst verschöbe jede spätere Korrektur der Rechenregel bereits ausgestellte Beträge.

4.3 Belege speichern Kunde, Aussteller und Preise als Snapshot

Ein Beleg hält Name und Anschrift des Kunden sowie die Preise der Positionen so fest, wie sie *zum Erstellzeitpunkt* galten. Wird die Anschrift eines Kunden im nächsten Jahr geändert, darf das die Rechnungen des Vorjahres nicht stillschweigend umschreiben. Diese Regel stand bereits in der ursprünglichen Spezifikation (`C-F-06`) und ist einer der Teile der Version 1.0, die jeden Umbau unverändert überstanden haben.

Für die eigene Firma fehlte dieselbe Regel lange: Jeder PDF-Export las das *aktuelle* Firmenprofil. Nach einem Umzug oder Bankwechsel hätte der erneute Export einer längst versendeten Rechnung plötzlich andere Absenderdaten getragen. Seit Version 3.0 speichert jeder Beleg auch den Aussteller als Snapshot (Flyway-Migration V4).

4.4 PDF und E-Rechnung müssen auf den Cent übereinstimmen

Version 1.0 rundete die Umsatzsteuer je Position und addierte die Ergebnisse. Das klingt harmlos und ist in fast jedem Test unauffällig — bis zehn Positionen zu je 1,10 € im Spiel sind: je Position gerundet ergibt das 2,10 € Steuer, je Steuersatz gerundet 2,09 €. Die Norm EN 16931 (Regel BR-CO-17) und die Bibliothek Mustang rechnen je Steuersatz; das PDF rechnete je Position. Dieselbe Rechnung hätte als PDF und als XML zwei verschiedene Endbeträge ausgewiesen.

Faktura rechnet deshalb je Steuersatz, weist Entgelt und Steuer je Satz im PDF aus (§ 14 Abs. 4 Nr. 7/8 UStG) und belegt die Übereinstimmung mit einem Test, der genau diesen Fall durch beide Ausgabewege schickt.

4.5 Die Oberfläche erfährt nur von Daten, die den Commit überlebt haben

Services veröffentlichen nach jeder schreibenden Operation ein `DatenGeaendertEreignis`, die Ansichten aktualisieren sich selbst. Sobald es echte Transaktionen gab, wurde der naive Zuhörer zum Fehler: Nach einem Rollback hätte die Oberfläche einen Datensatz angezeigt, den es nicht gibt. Die Brücke im `EreignisBus` lautet deshalb `@TransactionalEventListener(phase = AFTER_COMMIT, fallbackExecution = true)` — der Rückfallpfad hält Ereignisse aus Aufrufen ohne laufende Transaktion am Leben.

Darüber hinaus: Alles, was dauern kann (PDF, E-Rechnung, CSV, Datensicherung, Druck, Mailversand), läuft über einen Hintergrund-Executor. Zwischen-PDFs liegen in einem Sitzungsverzeichnis, das beim Beenden gelöscht wird. Und die Datensicherung zieht ihren Abzug über `VACUUM INTO`, statt eine Datei zu kopieren, an der offene Verbindungen und eine WAL-Seitendatei hängen.

5 Vorgehen

Zwölf Phasen in zwei Releases; die CI prüft jeden Push mit demselben Befehl wie lokal:

Release	Phasen	Inhalt
2.0	1–7	Qualitäts-Baseline (Validierung, Logging, CI, JaCoCo, SpotBugs) · Spring-IoC · SQLite + Flyway hinter den bestehenden Repository-Schnittstellen · vollständiger JavaFX-Neubau · Firmenprofil · E-Rechnung EN 16931 · MSI-Installer (jpackage) mit Tag-gesteuertem Release-Workflow
3.0	8–11	Transaktionale Nummernvergabe · Hintergrundausführung und sitzungsgebundene Zwischendateien · Datensicherung über `VACUUM INTO` · Belegausgabe als eigene Komponente · neue Oberfläche (Seitennavigation, Übersicht, Master-Detail, Statusabzeichen, Dunkelmodus) · Abdeckungsschwelle als Abbruchkriterium
3.0	12	Abschluss: systematische Fehlersuche über alle Komponenten, Steuer je Steuersatz, Steuernummer und Pflichtprofil, Aussteller-Snapshot, Zahlungseingang, Stornorechnung, eingebettete PDF-Schrift, plattformübergreifendes JAR, Oberflächendurchlauf mit Demodaten, Dokumentation auf Stand 3.0

Die Reihenfolge war bewusst gewählt: Die Qualitäts-Baseline kam *zuerst*, damit jeder folgende strukturelle Eingriff — Austausch der Persistenzschicht, Ersetzen des gesamten UI-Toolkits — eine Testsuite und eine statische Analyse *unter* sich hatte und nicht daneben.

6 Technologiewahl und Begründung

Ebene	Wahl	Begründung
Sprache	**Java 21**	LTS-Version; Records für die Wertetypen (`Summen`, `Kennzahlen`, `Steuerzeile`, …), Textblöcke für lesbares SQL
Verdrahtung	**Spring Boot**	Nicht wegen des Webservers, sondern wegen IoC-Container, deklarativem `@Transactional` und Ereignissystem. Transaktionsgebundene Ereignisse sind genau das, was die Nummernvergabe brauchte
Oberfläche	**JavaFX + AtlantaFX + Ikonli**	Lokaler Betrieb ist das Produktversprechen, also Desktop-Toolkit statt Browser. AtlantaFX liefert Theme-*Variablen*, dadurch trägt der Dunkelmodus ohne zweites Stylesheet
Datenhaltung	**SQLite + Spring JDBC + Flyway**	Ein Benutzer, keine Installation, eine Datei zum Sichern. PostgreSQL brächte einen zu betreibenden Dienst ohne Gegenwert. Flyway macht Schemaänderungen prüfbar; JDBC statt JPA, weil das Schema klein und explizites SQL leichter nachvollziehbar ist

Ebene	Wahl	Begründung
PDF	**Apache PDFBox**	Volle Kontrolle über ein festes kaufmännisches Layout; eingebettete Schrift (Liberation Sans), damit auch „Łukasz“ oder „Şahin“ korrekt erscheinen
E-Rechnung	**Mustang**	EN 16931 ist ein großer, validierter Standard — ZUGFeRD-CII-XML von Hand wäre ein eigenes Projekt
Austauschformat	**Jackson (JSON)**	Erhalten für die einmalige Übernahme und für Sicherungen; polymorphe Belegtypen über `@JsonTypeInfo`
Build und Qualität	**Maven** · **JUnit 5** · **JaCoCo** · **SpotBugs** · **GitHub Actions**	`verify` führt Tests, Abdeckungsschwelle und statische Analyse aus; die CI ruft denselben Befehl auf
Auslieferung	**jpackage**	MSI je Benutzer, dadurch weder Administratorrechte noch vorinstalliertes JDK nötig; daneben ein JAR mit den nativen JavaFX-Bibliotheken für Windows, Linux und macOS

Neue Produktpreise werden verlustfrei auf zwei Nachkommastellen normalisiert. Neue Belegsteuern werden je Satz mit `HALF_UP` gerundet; historische Beträge bleiben unverändert. Bezeichner, Kommentare, Spezifikationen und die gesamte Projektdokumentation sind deutsch — die Sprache der Fachlichkeit und ihrer Regularien.

7 Ergebnisse

- **Historischer Stand 3.0: 201 automatisierte Tests**, kein Fehlschlag; zur Abnahme der Version 1.0 waren es 71.
- **Abdeckungsschwelle im Build**: `verify` bricht unterhalb von 85 % Anweisungs- und 70 % Zweigabdeckung im nicht-grafischen Code ab. FXML-gebundene Ansichtsklassen sind ausgenommen, da ihre Dialogführung in eigenen GUI-freien Controller-Klassen liegt, die mitgezählt werden.
- **SpotBugs: keine Befunde**, und zwar erzwungen statt berichtet und ignoriert.
- **Performance wird am Produktionspfad geprüft.** SQLite-Suche, Belegliste, Dashboard, 50-Positionen-PDF und vollständiger ZIP-Export wurden lokal gemessen. Die drei gepackten Starts des ersten Laufs überschritten die Fünf-Sekunden-Grenze; zuletzt waren es zwei von drei. Damit ist die RC-Freigabe offen. Die früheren JSON-Tests belegen diese Startzeit nicht. Messwerte und Runner stehen im Modultestbericht, Kapitel 8.
- **Vollständige Nachvollziehbarkeit**: `Anforderungsabgleich.md` bildet jede Anforderung auf den umsetzenden Code und vorhandene Nachweise ab. Usability braucht echte Testpersonen; Netzwerk-Monitoring und externe Release-Prüfungen sind zusätzlich noch offen. Implementierter Code allein belegt diese Anforderungen nicht.
- **Veröffentlichung mit Freigabeschranken**: Ein Tag `v*` darf erst nach erfolgreichen Qualitäts-, Sicherheits-, Performance-, Plattform- und Installerprüfungen veröffentlichen. Für den aktuellen Kandidaten sind die externen Läufe noch offen.

8 Was ich gelernt habe

Ein Zähler im Speicher ist keine Geschäftsregel. „Lückenlos“ ist eine Eigenschaft der *Vergabe*, nicht der Nummern, die man zufällig in der Liste sieht. Die Umsetzung aus Version 1.0 wirkte in jedem Test korrekt, der nicht fehlschlug — genau das machte sie gefährlich. Die Lehre: Bei einer Invariante, die auch im Fehlerfall halten muss, *ist* der Test, der den Fehler erzwingt, die Anforderung. Alles andere ist Zierrat.

Ein Test, der den Fehler erzwingt, zahlt sich später noch einmal aus. Beim Abschluss bekam die Rechnungserstellung eine zweite Signatur mit Leistungsdatum; die alte wurde zur Default-Methode der Schnittstelle. Alles kompilierte, alle fachlichen Tests blieben grün — nur `NummernkreisIntegritaetTest` schlug an: Ein Aufruf über die Default-Methode läuft am Transaktions-Proxy von Spring vorbei, die Nummernvergabe landete in einer eigenen Transaktion, und ein gescheitertes Speichern hätte wieder eine Lücke hinterlassen. Ohne diesen einen Test wäre der Rückfall in genau den Fehler, den die Version 3.0 behoben hatte, unbemerkt ausgeliefert worden.

Autokonfiguration ist keine Zusicherung. Flyway lief unter Spring Boot 4 nicht allein dadurch, dass `flyway-core` im Klassenpfad lag; die Migration musste programmatisch im `dataSource`-Bean angestoßen werden. Heute prüfe ich, dass die Magie stattgefunden hat, bevor ich darauf aufbaue.

Rundung ist Fachlichkeit. Ob die Steuer je Position oder je Satz gerundet wird, sieht nach einem Implementierungsdetail aus und entscheidet doch, ob zwei gesetzlich geforderte Darstellungen derselben Rechnung übereinstimmen. Solche Stellen findet man nicht durch Lesen, sondern indem man beide Ausgabewege mit einem ungünstigen Beispiel vergleicht.

Schnittstellen machen einen Umbau überlebbar. Die Repository-Schnittstellen entstanden in Version 1.0 für JSON-Dateien. Der Wechsel auf SQLite verlangte neue Implementierungen und änderte darüber *nichts*. Dasselbe galt für die Oberfläche: Weil die Dialogführung in GUI-freien Controller-Klassen lag, ließ das vollständige Ersetzen von Swing durch JavaFX diese Klassen unberührt. Beide Gewinne waren nicht sichtbar, als die Grenzen gezogen wurden — das ist der Sinn des Ziehens.

Kompilieren und grüne Tests heißen nicht „funktioniert“. Reale Fehler — eine Kennzahlenkachel aus dem Fenster geschoben, abgeschnittene Tabellenspalten, Tastenkürzel, die nach dem Ansichtswechsel weiter den falschen Dialog öffneten, ein Detailbereich, der über seine Grenzen hinaus wuchs — waren für den Build unsichtbar und binnen Sekunden nach dem Start der Anwendung offensichtlich. Bei allem mit Oberfläche gehört das Hinsehen zur Definition von „fertig“ und nicht zur Nacharbeit.

Eine Abdeckungsschwelle soll nachziehen, nicht anstreben. Die Schwelle liegt knapp unter dem, was die Suite tatsächlich erreicht. Ihre Aufgabe ist, Rückschritte zu verhindern, nicht ein Ziel zu verkünden, nach dem niemand handelt; wer die Abdeckung hebt, zieht die Schwelle hinter sich nach.

Dokumentation verfällt schneller als Code — und leiser. Wenn ein Dokument veraltet, schlägt nichts fehl. Die Abschlusspräsentation beschreibt weiterhin Swing, manuelle DI und die E-Rechnung als Nichtziel — alle drei Aussagen stimmten in der Woche, in der die Folien entstanden. Spezifikationen brauchen dieselbe Prüfung auf Aktualität wie Code — und wo ein Dokument ein historischer Beleg ist, sollte es das offen ausweisen, statt stillschweigend als aktueller Stand gelesen zu werden.

8.1 Arbeiten mit Claude Code

Dieses Projekt wurde mit **Claude Code** als Werkzeug entwickelt. Es erscheint sinnvoller, das genau zu beschreiben, als es implizit zu lassen.

Was das Werkzeug gut konnte: mechanische Arbeit in einem Umfang, den ich von Hand nicht geleistet hätte — ein Paket über fast hundert Dateien hinweg umbenennen, Repository- und Service-Gerüste erzeugen, Spezifikationen und Testgerüste entwerfen, PlantUML-Quellen und Maven-Konfiguration schreiben, die Codebasis systematisch nach Fehlern durchsuchen. Es ist schnell darin, konsistente Struktur zu erzeugen, sobald die Struktur entschieden ist.

Was bei mir blieb: jede Entscheidung des Kapitels „Die Lösung“. Welche Regeln Invarianten sind, ob die Nummernvergabe in eine Transaktion gehört, was aus Version 1.0 bleibt und was verschwindet, welche fachlichen Lücken vor der Veröffentlichung geschlossen werden, und ob eine Änderung tatsächlich fertig war. Das Werkzeug schlug plausiblen Code vor; dass die Lückenlosigkeit die Anforderung ist, um die sich der gesamte Entwurf biegen muss, wusste es nicht — das steht im Lastenheft, nicht im Code.

Wo Korrektur nötig war: Ohne ausdrückliche Vorgabe fällt der Vorschlag generisch aus — der plausibel aussehende Zähler statt des transaktionalen. Fachliche Regeln müssen ausdrücklich benannt und im Ergebnis nachgeprüft werden, besonders in der Geschäftslogik.

Was das Vorgehen tragfähig machte: Änderungen im Zuschnitt einer Phase; CI, SpotBugs und Abdeckungsschwelle als ständige Leitplanke statt als Nacharbeit; und bei allem Sichtbaren: die Anwendung starten und hinsehen. Leitplanken wiegen schwerer, wenn Arbeit schnell entsteht — ein schneller Beitragender ohne Testsuite ist nur ein schneller Weg, Dinge zu zerbrechen.

9 Ausblick

- **Usability-Tests mit fünf Personen** — derzeit bewusst zurückgestellt; Q-05/Q-09 bleiben ohne empirischen Nachweis. Diese Grenze wird in der Vorführung benannt.
- **Steuerbefreiungen und Kleinunternehmerregelung (§ 19 UStG)** — heute werden 0-%-Positionen ohne Befreiungsgrund ausgewiesen; für den Einsatz durch Kleinunternehmer fehlt der Pflichthinweis auf der Rechnung.
- **Mahnwesen** — in Version 1.0 als Nichtziel geführt und der naheliegende nächste Schritt, nachdem Zahlungsziele und Zahlungseingänge existieren.
- **Installationspakete für Linux und macOS** — das JAR läuft bereits unter Windows, Linux und macOS (Apple Silicon); Intel-Macs bauen aus den Quellen. Nur der Installer ist heute Windows-spezifisch.

10 Ergänzung 3.1.0-rc.1 (10.10.2026)

Maßgeblicher RC-Umfang: EN 16931 CII mit eingebetteter Schema-/Schematron-Prüfung; XRechnung ist nicht implementiert. Lückenlose Nummernvergabe ist Projektinvariante GR-01, keine pauschale Aussage über gesetzliche Anforderungen oder Compliance. Historische Beträge und unbekannte Einheiten

werden erhalten; nicht reproduzierbares XML wird gesperrt. Details zu Snapshots, Migration, Einheiten, CSV und Rücksicherung stehen in Betrieb 3.1. Die v1.0-Abnahme bleibt ein historischer Nachweis. Aktuelle Ergebnisse, verbleibende Plattformprüfungen und offene Usability-Studien sind im Modultestbericht ausgewiesen. Frühere JSON-Zeitmessungen belegen weder SQLite-Performance noch die Startzeit des gepackten Programms.

10.1 Ergänzende Review- und Portfolio-Arbeit mit Codex

Die Review-Remediation 3.1.0-rc.1 wurde zusätzlich mit OpenAI Codex erarbeitet: Code- und Dokumentationsprüfung, Umsetzung der vereinbarten Korrekturen, Regressionen, Messungen und Aufbereitung der Ergebnisse. Dies ist KI-unterstützte Entwicklungsarbeit; die bestehenden Aussagen zur ursprünglichen Claude-Phase beschreiben nicht allein die Herkunft des heutigen Codes. Persönliche Eigenleistung sollte anhand tatsächlich getroffener Entscheidungen und selbst nachvollzogener Beispiele erläutert werden.

Der Portfolio-Schwerpunkt liegt auf Datenintegrität, nachvollziehbaren Fehlerfällen und einer reproduzierbaren Vorführung. Zusätzliche Produktfunktionen und die derzeit nicht realisierbare Usability-Studie sind zurückgestellt. Das Demo-Drehbuch und die Abschlussgrenzen sind maßgeblich für diese Darstellung; sie ersetzen keine technische RC-Freigabe.